缓存使用实践总结：以淘宝交易结算场景为例

原创 交易结算团队 大淘宝技术 2026年5月11日 01:29 浙江 158人



本文系统总结了汇金平台在缓存使用中的技术实践，涵盖**缓存理论**（适用条件、分类维度与选型方法）、**落地实践**（五类典型场景——配置/实体数据缓存、分布式锁、汇总计数等的选型依据与实现细节）以及**缓存安全**（穿透、击穿、雪崩、一致性、序列化、本地缓存并发等风险的成因与治理方案）。



谈起高并发与高性能总让人想到缓存，似乎缓存是一切性能问题的灵丹妙药，但真正开始实践不得不考虑如下问题：

1. 什么情况适合使用缓存？
2. 如何使用缓存并做好缓存选型？
3. 使用缓存时需要注意什么安全问题？

.....

本文主要内容：

1. 从诸多优秀实践抽象总结出缓存理论。
2. 以汇金平台为例介绍缓存理论到实践的过程。
3. 介绍进行缓存实践时需要注意的缓存安全事项。



缓存条件

缓存不是高并发的万能公式，待缓存数据应具备以下特征：

数据特征	要求	说明

数据不要求强一致性	必须	缓存与数据库是异构的，数据库事务对缓存不生效，而分布式事务改造成本过高（接受最终一致性）。
数据访问频率高	应该	若数据访问频率低，可考虑直接访问数据库。
数据更新频率低	应该	若数据更新频率高，缓存的命中率就会低，会影响缓存的使用效果。
数据访问要求高性能	适合	若数据访问无性能要求，可考虑直接访问数据库（前提是数据库能扛下压力）。

缓存应用

缓存分类

从不同的角度，缓存可划分为如下类型：

分类角度	缓存类型	缓存特征	缓存对象特征
存储端	本地缓存	缓存在应用机器上，缓存只对本机器可见	数据量小、机器不需数据共享、存在热点key
	分布式缓存	缓存在中心缓存中间件上，缓存对应用所有机器可见	数据量大、机器需要数据共享
存储介质	内存型缓存	缓存不进行持久化，存在丢失风险	缓存丢失无风险
	持久型缓存	缓存进行持久化，缓存淘汰或缓存容器重启不会丢失数据	缓存丢失代价大
存储时机	静态缓存	缓存不更新与失效，完全依赖于过期/重启失效后重刷	在线链路无更新逻辑
	旁路缓存	数据库更新后同步失效缓存	在线链路有更新逻辑
	直写缓存	在线链路直接更新缓存	在线链路有更新逻辑、更新非常频繁、汇总类数据

缓存选型

1. 基于缓存条件判断待缓存对象是否适合使用缓存。
2. 基于缓存分类表进行缓存选型：
   - 同分类角度的缓存类型互斥（本地缓存与分布式缓存例外能共存），不同分类角度的缓存类型可共存。
   - 根据待缓存对象特征从不同的分类角度去匹配不同的缓存类型，最终从三个角度组合出最终缓存选型方案。

对于缓存类型与数据特征匹配的技术细节：

Q1：为什么数据量小可选择本地缓存，数据量大需要考虑分布式缓存？

A1：缓存会占用内存空间，而单个应用机器的内存是有限且难以动态扩展的，如果数据量过大使用本地缓存可能会导致OOM，若限制了缓存空间的上限则可能导致淘汰频繁从而缓存命中率过低，而分布式缓存的空间是可横向动态扩展从而满足大数据量的需求。

Q2：为什么存在热点key需要考虑本地缓存？

A2：分布式缓存服务器为了确保机器可用性，会根据缓存key进行限流，当单key流量超出限流阈值则会拒绝访问缓存而直接访问数据库，对数据库造成巨大压力，这便是典型的缓存击穿案例。如果原本采用了分布式缓存，但出现缓存击穿问题需要治理，除了改造分布式缓存为本地缓存外，还可以保留分布式缓存并新增本地缓存以搭建二级缓存。

Q3：什么情况会导致缓存丢失代价大？

A3：大体上分为两种情况，一种是数据只存储于缓存服务器（该类数据一般为统计类数据，关系型实体数据不允许），另一种是缓存数据的初始化成本较大。

Q4：常见的缓存产品有哪些？

A4：一般而言，存储端与存储介质由缓存产品决定，存储时机是由应用具体编排设计。缓存产品主要是缓存框架与缓存中间件，开源主流缓存框架如Caffeine、Ehcache、Guava Cache等，集团内的缓存中间件主要是Tair MDB/LDB，其中MDB是内存型而LDB是持久型。



本文以汇金结算系统为例，结合常见的缓存场景介绍缓存使用实践：

场景一：经典缓存场景 – 缓存实例型配置类数据。

场景二：经典缓存场景 – 缓存实例型实体类数据。

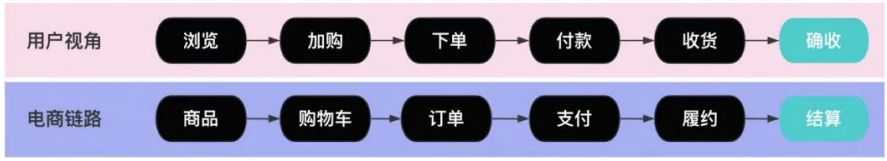
场景三：经典缓存场景 – 缓存单例型配置类数据。

场景四：分布式锁。

场景五：汇总计数。（单行数据热点更新）

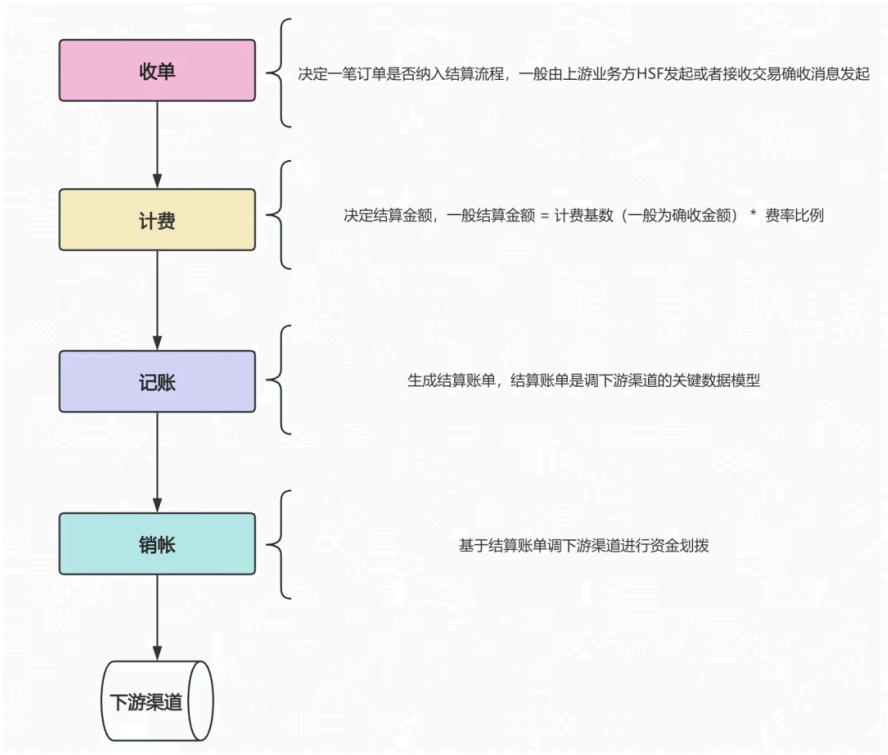
汇金结算基础介绍

交易链路下的结算全局视角：



费用项：每个业务需要在汇金接入费用项才能基于费用项触发结算流程。

一笔订单确收后，各个结算费用项会走一遍结算流程：收单->计费->记账->销帐。



场景一：缓存实例型配置类数据

数据特征：

1、实例型数据意味着数据量可能较大。

2、配置类数据意味着数据可能读频繁但写不频繁。

缓存选型：分布式缓存 + 内存型缓存 + 静态缓存

- 使用实践

缓存对象介绍

缓存的数据模型：汇金结算系统 – 计费费率实例。

费率实例：确定费率比例，费率实例是由费用项 + 商家ID + 费率因子（一般为订单标）路由得到。

费率实例具有以下特征：

1. 结算有非常多费用项，每个费用项对应非常多商家与多个费率因子，因此费率实例数量很大。

2. 体量较大的商家会频繁触发结算，同一条费率实例的访问非常频繁。
3. 费率实例属于配置类数据，其修改都是技术同学数据订正，因此修改频率非常低。

缓存理论指导：

缓存角度	缓存选型	选型依据
是否适合缓存	适合	费率实例属于配置类数据，对于体量较大的商家（也是流量大头）同一条费率的访问非常频繁，且数据更新非常低频。
存储端	分布式缓存	费率实例数据量庞大，本地缓存的缓存内存分配会有很大压力。
存储介质	内存型缓存	费率实例在缓存丢失无业务风险，始终以数据库为准。
存储时机	静态缓存	1、费率实例几乎不更新。 2、费率实例缓存只要求最终一致性。（因为费率实例有生效时间字段，只需等缓存自然过期重刷即可，而且非常紧急有实时性要求的话都可以人工重刷缓存）

场景二：缓存实例型实体类数据

数据特征：
1. 实例型数据意味着数据量可能较大。
2. 实体类数据意味着数据可能存在在线更新逻辑。

缓存选型：分布式缓存 + 内存型缓存 + 旁路缓存

- 使用实践

缓存对象介绍

缓存的数据模型：汇金结算系统 – CAE代扣协议签约信息。

CAE代扣协议签约信息：支付宝向集团侧提供了CAE代扣的结算能力，CAE代扣的前提是商家签约了CAE代扣协议，而CAE代扣协议的签约信息是落在支付宝侧，但汇金侧会在每次销帐调支付宝CAE代扣时快照存储一份CAE代扣签约信息。汇金更新CAE代扣签约信息时，会先查询

库中持久化值，对比持久化值与最新值，若签约状态相同则不做更新，若不同则以最新值为准更新落库。

CAE代扣协议签约信息具有以下特征：

1. 结算维护了多个代扣协议，每个代扣协议签约了非常多商家，因此签约信息数量很大。
2. 代扣协议有在线更新链路，商家的代扣签约信息会从未签约到已签约的在线链路更新。
3. 代扣协议的解约非常困难，只有在支付宝走周期较长的流程才能触发，因此一旦商家与平台签订了某个代扣协议，可以无限近似认为该签约信息是不会再变更了。

缓存理论指导：

缓存角度	缓存选型	选型依据
是否适合缓存	适合	CAE代扣协议签约信息是商家维度的，对于体量较大的商家（也是流量大头）同一条费率的访问非常频繁，且已签约协议几乎不更新。
存储端	分布式缓存	CAE代扣协议签约信息数据量庞大，本地缓存的缓存内存分配会有很大压力。
存储介质	内存型缓存	CAE代扣协议签约信息在缓存丢失无业务风险，始终以数据库为准。
存储时机	旁路缓存	CAE代扣协议签约信息有在线更新链路，需要在线更新后实时刷新缓存。

场景三：缓存单例型配置类数据

数据特征：

1. 单例型数据意味着数据量可能较小。
2. 配置类数据意味着数据可能读频繁但写不频繁。

缓存选型：本地缓存 + 内存型缓存 + 静态缓存

- 使用实践

缓存对象介绍

缓存的数据模型：汇金结算系统 – 收单网关配置。

收单网关配置：结算费用项有一份收单网关配置，该配置决定了一笔订单确收时是否需要收取该结算费用项。

收单网关配置具备以下特征：

1. 收单网关配置是单例型数据，与结算费用项是1:1关系，因此收单网关配置数据量非常小。
2. 收单网关配置是配置类数据，基本只有接入结算费用项时会变更，后续只能依靠人工订正几乎不修改。

缓存理论指导：

缓存角度	缓存选型	选型依据
是否适合缓存	适合	收单网关配置是费用项维度的，访问极其频繁且几乎不更新。
存储端	本地缓存	1、收单网关配置数据量非常小，不必使用分布式缓存。 2、访问极其频繁，本地缓存能减少网络IO提高查询效率。 3、无在线修改链路，不要求所有机器缓存数据一致，数据订正变更后分批重启机器还具备配置生效的灰度能力。
存储介质	内存型缓存	CAE代扣协议签约信息在缓存丢失无业务风险，始终以数据库为准。
存储时机	静态缓存	1、收单网关配置无在线更新链路，几乎不更新。 2、收单网关配置只要求最终一致性。

场景四：缓存互斥量：分布式锁

数据特征：

互斥量为单一简单数据，无特别的特征，主要利用互斥量对业务流程加锁。

缓存选型：分布式缓存 + 持久型缓存 + 直写缓存

- 使用实践

缓存的数据模型：信号量（业务自定义常量）。

基本实现：基于Tair实现分布式锁

缓存理论指导：

缓存角度	缓存选型	选型依据

是否适合缓存	适合	分布式锁要求加锁解锁高性能。
存储端	分布式缓存	分布式锁只能用分布式缓存，所有机器需要共享锁数据。
存储介质	持久型缓存	分布式锁不允许锁数据丢失，锁数据仅存储于数据中，丢失有并发更新风险。
存储时机	直写缓存	在线链路有更新逻辑，更新非常频繁。

场景五：缓存汇总类数据：汇总计数

数据特征：

1. 汇总类数据意味着数据量可能较小。
2. 汇总类数据意味着可能存在频繁的单行热点更新。

缓存选型：分布式缓存 + 持久型缓存 + 直写缓存

- 使用实践 – 库存扣减

最常见的场景是库存扣减，核心要点如下：

1. 一致性容忍：大部分库存扣减接受一定程度的有损，能接受少卖但不接受超卖。
2. 扣减流程：先进行缓存扣减，再插入数据库扣减流水，该顺序保证了不会超卖。
3. 缓存能力：利用了LDB的持久化与原子扣减能力，扣减时可以设置扣减下限。
4. 分桶设计：库存在缓存中分桶存储，扣减时路由其一分桶进行扣减，有专门分桶管理模块。

缓存理论指导：

缓存角度	缓存选型	选型依据
是否适合缓存	适合	库存访问非常频繁，且访问有高性能要求，同时热点更新数据库压力过大。
存储端	分布式缓存	库存数据量大，所有机器需要共享库存数据。

存储介质	持久型缓存	库存缓存数据初始化成本大，且库存缓存数据需要实时更新。
存储时机	直写缓存	在线链路有更新逻辑，更新非常频繁。

- 使用实践 – 结算计费累积量

缓存对象介绍

缓存的数据模型：汇金结算系统 – 计费累积量。

计费累积量：部分费用项需要阶梯计费，根据某个数据累积值（如GMV）分阶梯确定费率比例，累积值存储于累积量。

计费累积量备以下特征：

1. 累积量的维度由具体费用项确定，数据量可能多也可能少，视具体费用项而定。
2. 读取非常频繁，部分业务每次计费都会读取到同一条累积量。
3. 更新非常频繁，且存在单行热点更新，部分业务的热点更新非常剧烈，频繁导致数据库活跃连接数过高。

缓存理论指导：

缓存角度	缓存选型	选型依据
是否适合缓存	适合	累积量访问非常频繁，且访问有高性能要求，同时热点更新数据库压力过大。
存储端	分布式缓存	累积量数据量大，所有机器需要共享累积量数据。
存储介质	持久型缓存	累积量缓存数据初始化成本大，且累积量缓存数据需要实时更新。
存储时机	直写缓存	在线链路有更新逻辑，更新非常频繁。



风险场景

缓存主要有如下表风险场景：（某些场景定义实际在业界仍存在多种口径，以下分类尽量做到正交）

风险场景	场景定义
缓存穿透	待查询数据在业务语义上不存在，数据库必然不存在该数据，每次请求都会穿透缓存直接查询数据库。
缓存击穿	待查询数据在业务语义上会存在，大量请求到缓存服务器返回Miss后直接查询数据库。
缓存雪崩	查询请求大面积地广泛地直接查询数据库，缓存近似失去了其数据库保护能力。
缓存一致性问题	由于缓存与数据库数据不一致或者各个机器缓存数据不一致进而引发的问题。
缓存序列化问题	缓存序列化方式变更，反序列化失败导致缓存数据获取错误。
本地缓存并发问题	本地缓存值的原地修改对所有线程可见，当修改应为线程私有时则错误地让所有线程可见。

风险原因与治理

针对不同的风险场景，总结梳理了主要原因与应对方案：

风险场景	主要原因	应对方案
缓存穿透	1、前端系统存在业务预期外的访问路径。 2、后端系统未过滤业务预期外的访问参数。 3、恶意流量访问。	1、增加布隆过滤器，执行查询前先确认数据是否存在。 2、空值缓存。
缓存击穿	1、热点KEY触发缓存服务器限流。 2、缓存过期。 3、缓存被淘汰： • 缓存容量过小。 • 缓存淘汰策略不合理。	1、热点KEY问题： • 从业务角度把热点KEY打散。 • 把分布式缓存改为本地缓存。 • 在分布式缓存的基础上针对热点KEY进行本地缓存。 2、合理设置缓存过期时间。

		3、申请足够的缓存容量，考虑选择LRU淘汰策略。
缓存雪崩	1、大面积缓存过期。 2、大面积地缓存穿透与缓存击穿。 3、缓存服务器不可用。 4、缓存服务器切换。（新机器可能无数据）	1、打散缓存过期时间，避免同一时间集中过期过期。 2、持续治理缓存穿透与缓存击穿问题。 3、缓存集群要有主从架构，具备容灾能力。 4、缓存服务器切换前先做数据预热。
缓存一致性问题	1、数据库更新后，缓存未及时更新。 2、机器需要共享缓存数据却使用本地缓存。	1、对于旁路与直写缓存，确保代码能准确重刷缓存；对于静态缓存，数据库订正后确保在业务预期时间内让缓存失效。 2、使用本地缓存时，需要充分考虑机器是否共享缓存数据。
缓存序列化问题	1、缓存序列化器变更（或其版本变更）。 2、缓存数据结构体变更。	1、涉及序列化变更不能一把切，需要做新老版本兼容，例如对于老版本数据使用老序列化方式，新版本数据使用新序列化方式。
本地缓存并发问题	1、缓存对象有在线修改逻辑，但却使用了本地缓存的同时未进行克隆。 （热点KEY治理使用本地缓存时尤其要注意该问题，因为分布式缓存每次反序列化的都是新对象不存在该问题，缓存数据可能有在线更新逻辑而未注意）	1、对于需要线程私有修改的本地缓存值（或直接全局生效），获取本地缓存值时先进行克隆，后续修改则在线程快照值修改（有性能开销）。 2、对于本地缓存限制不可修改，修改应在后续转换的某类DTO。



本文作者觸弦，来自淘天集团–交易结算团队。我们是负责淘天资金结算的交易结算团队，承载电商交易中资金流的精准、安全、高效流转。覆盖买家支付结算、商家货款结算、平台佣金清分、退款资金回流、跨境汇率结算、账务核对与对账等淘天电商资金全场景。我们正构建AI驱动的资金结算新范式，实现“业财一体”的智能化升级，让每一分钱都"算得清、流得准、管得住"。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法



大淘宝技术

大淘宝技术官方账号

967篇原创内容

公众号

阅读 6301

留言

写留言

还没有留言，来写首评